

PRÁCTICA 1

Control de Iluminación RGB con NeoPixel en ESP32-C6

Sistemas Embebidos

MicroPython

1. Objetivo

El objetivo de esta práctica es implementar un sistema de control de iluminación RGB sobre un microcontrolador ESP32-C6 programado con MicroPython utilizando un LED NeoPixel WS2812. De manera complementaria, se integran los siguientes conceptos:

- Configuración de entradas digitales mediante GPIO.
- Uso de resistencias internas PULL-UP y PULL-DOWN.
- Lectura de botones mediante polling.
- Control de colores RGB mediante tecnología NeoPixel.
- Implementación de secuencias de transición de color.
- Aplicación del modo de bajo consumo LIGHTSLEEP.
- Uso de diccionarios para asociar botones con funciones.

1.1 Objetivos Específicos

- Configurar tres botones digitales utilizando resistencias internas un PULL-UP y dos PULL-DOWN.
- Controlar un LED NeoPixel conectado al ESP32-C6.
- Implementar un modo de color fijo activado mediante BTN1.
- Implementar una secuencia automática de transición RGB mediante BTN2.
- Aplicar el modo LIGHTSLEEP mediante BTN3 para reducir el consumo energético.
- Organizar el firmware utilizando estructuras de datos que permitan asociar entradas físicas con acciones específicas.

1.2 Alcance de la Práctica

El sistema resultante demuestra la integración de entradas digitales, control de un LED RGB y gestión básica de energía en una plataforma de sistemas embebidos. El alcance comprende exclusivamente la implementación en firmware con MicroPython sobre ESP32-C6, la lectura de botones y el control de un NeoPixel.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontrolador principal
LED RGB integrado a la placa	1	LED RGB barrido de colores
Push Button	3	Entradas de usuario
Resistencias internas	3	PULL-UP y PULL-DOWN configuradas por software

2.2 Mapa de Pines

Variable	GPIO ESP32-C6	Resistencia Interna	Función
BTN1	GPIO 3	PULL_DOWN	Color fijo
BTN2	GPIO 4	PULL_DOWN	Secuencia RGB
BTN3	GPIO 1	PULL_UP	Modo LIGHTSLEEP
NeoPixel	GPIO 8	—	Salida RGB

2.3 Esquema de Conexión

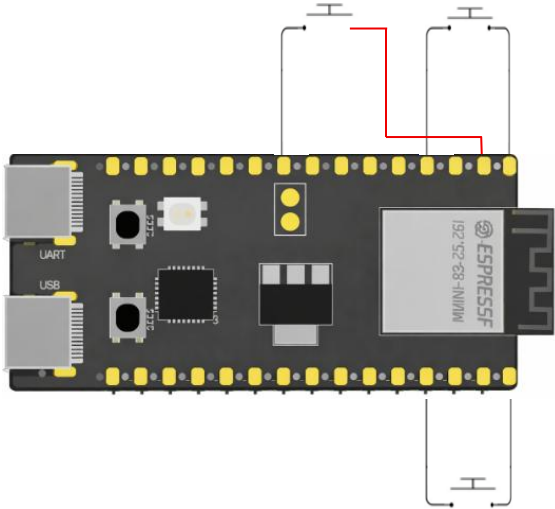


Figura 1. Esquema general del sistema (Push Buttons, tierra y 3.3 V)

2.4 Pruebas Eléctricas Realizadas

Antes de cargar el firmware se realizaron las siguientes verificaciones eléctricas para garantizar la correcta conexión de los componentes y prevenir posibles daños al microcontrolador:

- **Verificación de alimentación del ESP32-C6:** Se midió con multímetro la diferencia de potencial entre el pin de 3.3 V y GND de la placa, obteniéndose aproximadamente 3.3 V. Esta medición confirmó que la fuente de alimentación y el regulador interno del ESP32-C6 funcionaban correctamente.
- **Prueba de continuidad en botones con configuración PULL-DOWN:** Para los botones conectados a los GPIO 3 y 4, se verificó mediante la función de continuidad del multímetro que uno de sus terminales estuviera correctamente conectado a tierra (GND). Asimismo, se comprobó la continuidad entre el terminal opuesto de cada botón y su respectivo GPIO, garantizando una conexión eléctrica adecuada para la detección de pulsaciones.
- **Prueba de continuidad en botón con configuración PULL-UP:** Para el botón conectado al GPIO 1, se verificó la continuidad entre uno de sus terminales y la línea de 3.3 V. De igual manera, se comprobó la continuidad entre el terminal restante y el GPIO correspondiente, validando la correcta implementación de la configuración PULL-UP utilizada en el circuito.
- **Verificación de funcionamiento de los pulsadores:** Se comprobó mediante el multímetro que cada push button presentara continuidad eléctrica entre sus terminales al ser presionado, confirmando el cierre correcto del circuito. De igual forma, al liberar el pulsador se verificó la ausencia de continuidad (circuito abierto), validando el funcionamiento mecánico y eléctrico de cada botón.
- **Verificación general de conexiones:** Se revisó que no existieran circuitos abiertos entre los botones y los pines asignados del ESP32-C6, asegurando que cada entrada digital pudiera cambiar de estado lógico cuando se accionara el pulsador.

Todas las mediciones fueron realizadas con multímetro digital antes de energizar completamente el sistema, siguiendo buenas prácticas de prototipado para sistemas embebidos.

3. Arquitectura del Software

El proyecto está organizado en un archivo de micropython:

Archivo	Responsabilidad
p3.py	Configuración GPIO, control NeoPixel, gestión de botones y modo sleep

4. Descripción Técnica Detallada

4.1 Lógica principal

Este es el archivo central del proyecto. Integra todos los subsistemas del sistema embebido: configuración de GPIO, lectura de botones, control de un LED RGB direccionable NeoPixel y gestión de energía mediante el modo LIGHTSLEEP del ESP32-C6. El programa utiliza tres pulsadores para seleccionar diferentes modos de operación: un modo de color fijo, una secuencia automática de transición entre colores y un modo de bajo consumo.

La detección de los botones se realiza mediante polling continuo dentro del bucle principal, mientras que el control del NeoPixel se lleva a cabo modificando los valores de los canales rojo, verde y azul (RGB). Adicionalmente, el código emplea diccionarios para organizar la configuración de hardware y asociar cada botón con la función correspondiente, facilitando la escalabilidad y el mantenimiento del sistema.

```
from machine import Pin,lightsleep
import time
import neopixel

pines = {}
btns = {
    "b1": {"pin": 3, "pull": Pin.PULL_DOWN, "activo": 1},
    "b2": {"pin": 4, "pull": Pin.PULL_DOWN, "activo": 1},
    "b3": {"pin": 1, "pull": Pin.PULL_UP, "activo": 0},
}

colores = [0,0,0]

def def_color_fijo():
    colores[0] = 0 if colores[0] else 255
    pines["rgb"][0] = colores
    pines["rgb"].write()

fase = 0
def def_secuencia():
    global fase
    step = 10
    if fase == 0: # R -> Y
        colores[1] += step
        if colores[1] >= 255:
            colores[1] = 255
            fase = 1
    elif fase == 1: # Y -> G
        colores[0] -= step
        if colores[0] <= 0:
            colores[0] = 0
            fase = 2
    elif fase == 2: # G -> C
        colores[2] += step
        if colores[2] >= 255:
            colores[2] = 255
            fase = 3
    elif fase == 3: # C -> B
        colores[1] -= step
        if colores[1] <= 0:
            colores[1] = 0
            fase = 4
```

```

        elif fase == 4: # B -> M
            colores[0] += step
            if colores[0] >= 255:
                colores[0] = 255
                fase = 5
        elif fase == 5: # M -> R
            colores[2] -= step
            if colores[2] <= 0:
                colores[2] = 0
                fase = 0
        pines["rgb"][0] = colores
        pines["rgb"].write()

def def_sleep():
    global colores
    colores = [0,0,0]
    pines["rgb"][0] = colores
    pines["rgb"].write()
    lightsleep()
funciones = {
    "b1": def_color_fijo,
    "b2": def_secuencia,
    "b3": def_sleep
}

def configIO():
    for nombre, cfg in btns.items():
        pines[nombre] = Pin(cfg["pin"], Pin.IN, cfg["pull"])
    pines["rgb"] = neopixel.NeoPixel(Pin(8),1)
configIO()

def leerBtn():
    for nombre, cfg in btns.items():
        if pines[nombre].value() == cfg["activo"]:
            return nombre
    return None

btn = None
while True:
    lectura = leerBtn()
    if lectura is not None:
        btn = lectura
        colores = [0,0,0]
    if btn in btns:
        funciones[btn]()
        time.sleep_ms(100)

```

4.1.1 Organización del código

Elemento	Responsabilidad
machine.Pin	Configuración y lectura de GPIO
machine.lightsleep	Modo de bajo consumo del ESP32-C6
neopixel	Control del LED RGB direccionable
time	Temporización mediante retardos
configIO()	Inicialización de botones y NeoPixel
leerBtn()	Detección del botón presionado
def_color_fijo()	Cambio entre LED apagado y rojo fijo
def_secuencia()	Generación de transición automática de colores
def_sleep()	Entrada al modo LIGHTSLEEP
while True	Ejecución continua del sistema

4.1.2 Configuración de GPIO, NeoPixel y diccionario de pines

El programa utiliza un diccionario denominado `pines` para almacenar dinámicamente todos los objetos de hardware utilizados durante la ejecución.

```
pines = {}
```

La instrucción anterior crea un diccionario vacío. Posteriormente, durante la función `configIO()`, se agregan a este diccionario los objetos correspondientes a los botones y al LED NeoPixel.

Por ejemplo:

```
pines[nombre] = Pin(cfg["pin"], Pin.IN, cfg["pull"])
```

Almacena cada botón utilizando como clave su nombre lógico (`b1`, `b2` o `b3`).

La configuración de los pulsadores se almacena en el diccionario `btns`, el cual centraliza toda la información necesaria para inicializar y leer cada botón del sistema.

```
btns = {
    "b1": {"pin": 3, "pull": Pin.PULL_DOWN, "activo": 1},
    "b2": {"pin": 4, "pull": Pin.PULL_DOWN, "activo": 1},
    "b3": {"pin": 1, "pull": Pin.PULL_UP, "activo": 0},
}
```

Cada entrada del diccionario representa un botón físico y contiene tres parámetros:

- **pin:** GPIO del ESP32-C6 al que se encuentra conectado el pulsador.
- **pull:** tipo de resistencia interna utilizada para fijar un estado lógico estable cuando el botón no está presionado.

- **activo:** nivel lógico que indica una pulsación válida.

Por ejemplo:

```
"b1": {"pin": 3, "pull": Pin.PULL_DOWN, "activo": 1},
```

indica que el botón **b1** está conectado al GPIO 3 y utiliza una resistencia interna **PULL_DOWN**. En esta configuración el pin permanece normalmente en nivel lógico bajo (0 V) y cambia a nivel alto (3.3 V) cuando el botón es presionado.

Adicionalmente, se define la variable global:

```
colores = [0,0,0]
```

La lista colores almacena los valores RGB del LED NeoPixel. Inicialmente todos los valores se establecen en cero, lo que significa que el LED comienza apagado:

```
[0,0,0]
```

A medida que el programa se ejecuta, los distintos modos de operación modifican estos valores para generar colores fijos o transiciones automáticas en el NeoPixel.

4.1.3 Modo de Color Fijo

La función `def_color_fijo()` implementa un modo de operación que alterna el estado del canal rojo del LED NeoPixel entre apagado y encendido máximo.

```
def def_color_fijo():
    colores[0] = 0 if colores[0] else 255
    pines["rgb"][0] = colores
    pines["rgb"].write()
```

La primera instrucción:

```
colores[0] = 0 if colores[0] else 255
```

utiliza una expresión condicional para modificar el valor almacenado en el canal rojo de la lista colores. El índice 0 corresponde al componente rojo (R) del modelo RGB.

Su funcionamiento es el siguiente:

- Si `colores[0]` contiene un valor distinto de cero (el color rojo está encendido), se asigna el valor 0, apagando dicho canal.
- Si `colores[0]` vale 0 (el color rojo está apagado), se asigna el valor 255, correspondiente a la máxima intensidad luminosa.

De esta manera, cada vez que se ejecuta la función se produce una conmutación entre los estados ON y OFF del color rojo.

Posteriormente, la instrucción:

```
pin["rgb"][0] = colores
```

copia los valores RGB almacenados en la lista colores hacia el primer LED del objeto NeoPixel. El índice [0] indica que se está configurando el único LED presente en la tira o módulo NeoPixel utilizado en el proyecto.

Finalmente:

```
pin["rgb"].write()
```

transmite los nuevos valores RGB al dispositivo físico. Es importante destacar que la asignación realizada en la línea anterior únicamente modifica los datos almacenados en memoria; el cambio visual en el LED ocurre hasta que se ejecuta el método write(), el cual envía la información mediante el protocolo de comunicación del NeoPixel.

Como resultado, cada activación de esta función alterna el LED entre apagado ([0,0,0]) y rojo intenso ([255,0,0]).

4.1.4 Secuencia Automática de Colores

La función def_secuencia() implementa una transición gradual entre diferentes colores del espectro RGB. Para ello utiliza una variable global denominada fase, la cual indica la etapa actual de la secuencia.

```
fase = 0

def def_secuencia():
    global fase
```

La instrucción global fase permite que la función modifique directamente la variable global definida fuera de ella. De esta forma, el sistema recuerda en qué punto de la secuencia se encontraba durante la ejecución anterior.

Al inicio de la función se define:

```
step = 10
```

La variable step representa la cantidad de unidades que se incrementará o decrementará un canal RGB en cada ejecución de la función. Esto permite generar cambios graduales de color en lugar de transiciones bruscas.

La secuencia completa está formada por seis fases:

Fase 0: Rojo → Amarillo

```
if fase == 0:
    colores[1] += step
```

Durante esta etapa el canal rojo permanece encendido mientras el canal verde aumenta progresivamente. Como resultado, el color cambia desde rojo puro ([255,0,0]) hasta amarillo ([255,255,0]).

Cuando el canal verde alcanza su valor máximo:

```
if colores[1] >= 255:
    colores[1] = 255
    fase = 1
```

se limita a 255 y la secuencia avanza a la siguiente fase.

Fase 1: Amarillo → Verde

```
elif fase == 1:
    colores[0] -= step
```

En esta etapa el canal verde permanece al máximo y el canal rojo disminuye gradualmente. El color evoluciona desde amarillo ([255,255,0]) hasta verde ([0,255,0]).

Una vez que el rojo llega a cero, la variable fase cambia a 2.

Fase 2: Verde → Cian

```
elif fase == 2:
    colores[2] += step
```

El canal azul comienza a aumentar mientras el verde permanece encendido. Esto genera una transición desde verde ([0,255,0]) hasta cian ([0,255,255]).

Al alcanzar el valor máximo del canal azul, se cambia a la fase 3.

Fase 3: Cian → Azul

```
elif fase == 3:
    colores[1] -= step
```

Durante esta etapa disminuye progresivamente el canal verde mientras el azul permanece encendido. El color cambia desde cian ([0,255,255]) hasta azul ([0,0,255]).

Al finalizar, la secuencia avanza a la fase 4.

Fase 4: Azul → Magenta

```
elif fase == 4:
    colores[0] += step
```

El canal rojo aumenta gradualmente mientras el azul permanece al máximo. Esto produce una transición desde azul ([0,0,255]) hasta magenta ([255,0,255]).

Cuando el rojo alcanza su valor máximo, la variable fase cambia a 5.

Fase 5: Magenta → Rojo

```
elif fase == 5:  
    colores[2] -= step
```

Finalmente, el canal azul disminuye progresivamente mientras el rojo permanece encendido. El color evoluciona desde magenta ([255,0,255]) hasta rojo ([255,0,0]).

Cuando el azul llega a cero:

```
if colores[2] <= 0:  
    colores[2] = 0  
    fase = 0
```

la secuencia vuelve a la fase inicial, permitiendo que el ciclo de colores se repita indefinidamente.

Una vez calculados los nuevos valores RGB, éstos se envían al NeoPixel mediante:

```
pinnes["rgb"][0] = colores  
pinnes["rgb"].write()
```

La primera instrucción actualiza los valores almacenados para el LED, mientras que write() transmite físicamente la información al NeoPixel. Como resultado, cada llamada a def_secuencia() genera un pequeño avance dentro del ciclo cromático completo, produciendo una animación continua y suave de colores.

4.1.5 Modo de Bajo Consumo y Asociación de Funciones

La función def_sleep() implementa el modo de ahorro de energía del sistema utilizando la función lightsleep() del módulo machine. Antes de colocar al ESP32-C6 en estado de bajo consumo, la función apaga completamente el LED NeoPixel para evitar consumo innecesario de corriente y garantizar que el sistema entre en reposo con todos los indicadores visuales desactivados.

```
def def_sleep():  
    global colores  
    colores = [0,0,0]  
    pinnes["rgb"][0] = colores  
    pinnes["rgb"].write()  
    lightsleep()
```

La instrucción global colores permite modificar la variable global que almacena el estado actual del LED RGB. Posteriormente, la lista colores se reinicializa con los valores [0,0,0], lo que equivale a apagar los tres canales de color (rojo, verde y azul).

A continuación, la nueva configuración de color se asigna al NeoPixel mediante:

```
pinnes["rgb"][0] = colores
```

y se transmite físicamente al LED utilizando:

```
pines["rgb"].write()
```

Finalmente, la instrucción:

```
lightsleep()
```

coloca al ESP32-C6 en modo **LIGHTSLEEP**, un estado de bajo consumo en el que el procesador suspende gran parte de su actividad para reducir el gasto energético. Durante este modo, el microcontrolador permanece en espera hasta que ocurre un evento capaz de despertarlo, como una señal externa en un GPIO configurado para ello.

Posteriormente se define el diccionario funciones:

```
funciones = {  
    "b1": def_color_fijo,  
    "b2": def_secuencia,  
    "b3": def_sleep  
}
```

Este diccionario actúa como una **tabla de asociación entre botones y funciones**. Cada clave corresponde al identificador de un botón y cada valor almacena una referencia a la función que debe ejecutarse cuando dicho botón es detectado por el sistema.

4.1.6 Configuración de Entradas y Salidas

La función configIO() se encarga de inicializar todos los dispositivos de entrada y salida utilizados por el sistema. Su objetivo es configurar los botones físicos definidos en el diccionario btns y crear el objeto encargado de controlar el LED RGB NeoPixel.

```
def configIO():  
    for nombre, cfg in btns.items():  
        pines[nombre] = Pin(cfg["pin"], Pin.IN, cfg["pull"])  
    pines["rgb"] = neopixel.NeoPixel(Pin(8),1)  
  
configIO()
```

La instrucción:

```
for nombre, cfg in btns.items():
```

recorre cada uno de los elementos almacenados en el diccionario btns. En cada iteración:

- nombre contiene el identificador lógico del botón ("b1", "b2" o "b3").
- cfg contiene el diccionario de configuración asociado a dicho botón.

Por ejemplo, para el botón b1, la variable cfg contiene:

```
{
  "pin": 3,
  "pull": Pin.PULL_DOWN,
  "activo": 1
}
```

Posteriormente, se crea un objeto Pin para cada botón mediante:

```
pines[nombre] = Pin(cfg["pin"], Pin.IN, cfg["pull"])
```

En esta instrucción:

- `cfg["pin"]` indica el número de GPIO utilizado.
- `Pin.IN` configura el pin como entrada digital.
- `cfg["pull"]` activa la resistencia interna correspondiente (`PULL_UP` o `PULL_DOWN`).

Cada objeto Pin creado se almacena dentro del diccionario `pines`, utilizando como clave el nombre lógico del botón. Como resultado, el diccionario queda estructurado de forma similar a:

```
pines = {
  "b1": <Pin GPIO3>,
  "b2": <Pin GPIO4>,
  "b3": <Pin GPIO1>
}
```

Una vez configurados los botones, se inicializa el LED RGB NeoPixel mediante:

```
pines["rgb"] = neopixel.NeoPixel(Pin(8), 1)
```

La clase `NeoPixel()` crea un objeto capaz de controlar LEDs direccionables WS2812 o compatibles. Los parámetros utilizados indican:

- `Pin(8)`: GPIO utilizado para transmitir los datos seriales al LED.
- `1`: número de LEDs NeoPixel conectados.

El objeto resultante también se almacena dentro del diccionario `pines` utilizando la clave `"rgb"`, centralizando así el acceso a todos los periféricos del sistema en una única estructura de datos.

Finalmente, la instrucción:

```
configIO()
```

Ejecuta la función inmediatamente después de su definición, garantizando que todos los GPIO y el LED NeoPixel queden correctamente configurados antes de que el programa entre en funcionamiento. Esto asegura que las lecturas de los botones y el control del LED puedan realizarse desde el inicio de la ejecución del firmware.

4.1.7 Lectura de Botones

Una vez configurados los GPIO mediante `configIO()`, el sistema utiliza la función `leerBtn()` para identificar qué botón se encuentra presionado en cada instante. Esta función implementa un mecanismo de **polling**, es decir, una consulta periódica y continua del estado de las entradas digitales.

```
def leerBtn():
    for nombre, cfg in btns.items():
        if pines[nombre].value() == cfg["activo"]:
            return nombre
    return None
```

La instrucción:

```
for nombre, cfg in btns.items():
```

recorre todos los botones definidos en el diccionario `btns`. Durante cada iteración se obtiene el nombre lógico del botón (`b1`, `b2` o `b3`) y su configuración asociada.

Posteriormente se evalúa la condición:

```
if pines[nombre].value() == cfg["activo"]:
```

El método `value()` pertenece a la clase `Pin` y permite leer el estado lógico actual del GPIO configurado como entrada. Dependiendo del tipo de resistencia interna utilizada, un botón puede considerarse activo cuando el pin toma el valor lógico 1 o el valor lógico 0.

Por esta razón, cada botón posee el parámetro "activo" dentro de su configuración:

```
"b1": {"pin": 3, "pull": Pin.PULL_DOWN, "activo": 1}
"b2": {"pin": 4, "pull": Pin.PULL_DOWN, "activo": 1}
"b3": {"pin": 1, "pull": Pin.PULL_UP, "activo": 0}
```

De esta manera:

- Los botones **b1** y **b2** utilizan resistencias **PULL_DOWN**, por lo que se consideran presionados cuando el GPIO cambia a nivel lógico **1**.
- El botón **b3** utiliza una resistencia **PULL_UP**, por lo que se considera presionado cuando el GPIO cambia a nivel lógico **0**.

Si se detecta un botón activo, la función retorna inmediatamente su identificador:

```
return nombre
```

Por ejemplo, si el usuario presiona el botón conectado al GPIO 4, la función devolverá:

```
"b2"
```

Si ningún botón se encuentra activo después de recorrer todo el diccionario, se ejecuta:

```
return None
```

indicando que no existe ninguna pulsación válida en ese momento.

Este mecanismo permite desacoplar la detección física de los botones de las acciones que posteriormente ejecutará el programa, ya que el bucle principal únicamente necesita consultar el valor retornado por leerBtn() para determinar qué función asociada debe ejecutarse.

4.1.7 Bucle Principal del Sistema

El funcionamiento general del programa se desarrolla dentro de un bucle infinito while True, encargado de supervisar continuamente el estado de los botones y ejecutar la función asociada al último botón detectado.

```
btn = None

while True:
    lectura = leerBtn()

    if lectura is not None:
        btn = lectura
        colores = [0,0,0]

    if btn in btns:
        funciones[btn]()
        time.sleep_ms(100)
```

Antes de iniciar el bucle principal se declara la variable:

```
btn = None
```

Esta variable almacena el identificador del botón actualmente seleccionado. Inicialmente se asigna el valor None, indicando que todavía no se ha detectado ninguna pulsación.

Dentro del bucle infinito, la primera operación consiste en llamar a la función leerBtn():

```
lectura = leerBtn()
```

Como se explicó anteriormente, esta función realiza un barrido de todos los botones configurados y retorna el nombre del botón activo ("b1", "b2" o "b3"). Si ningún botón se encuentra presionado, retorna None.

Posteriormente se evalúa la condición:

```
if lectura is not None:
```

Cuando se detecta una pulsación válida, el identificador retornado por leerBtn() se almacena en la variable btn:

```
btn = lectura
```

De esta forma, el sistema recuerda qué modo de funcionamiento debe ejecutar a partir de ese momento.

A continuación, se reinicia el estado del LED RGB mediante:

```
colores = [0,0,0]
```

Esto garantiza que, al cambiar de modo de operación, la nueva función comience siempre desde una condición conocida. Por ejemplo, si el usuario cambia del modo de secuencia automática al modo de color fijo, el LED se apaga momentáneamente antes de aplicar el nuevo comportamiento.

Posteriormente se verifica:

```
if btn in btns:
```

Esta condición comprueba que la variable btn contenga un identificador válido definido dentro del diccionario btns. Si la condición se cumple, el programa accede al diccionario funciones y ejecuta dinámicamente la función asociada:

```
funciones[btn] ()
```

Gracias a esta técnica, el programa puede invocar diferentes funciones utilizando únicamente el nombre del botón almacenado en btn.

Por ejemplo:

```
btn = "b1"  
funciones[btn] ()
```

equivale a ejecutar:

```
def_color_fijo()
```

De manera similar:

```
btn = "b2"
```

ejecutará:

```
def_secuencia()
```

y

```
btn = "b3"
```

ejecutará:

```
def_sleep()
```

Finalmente se introduce un retardo mediante:

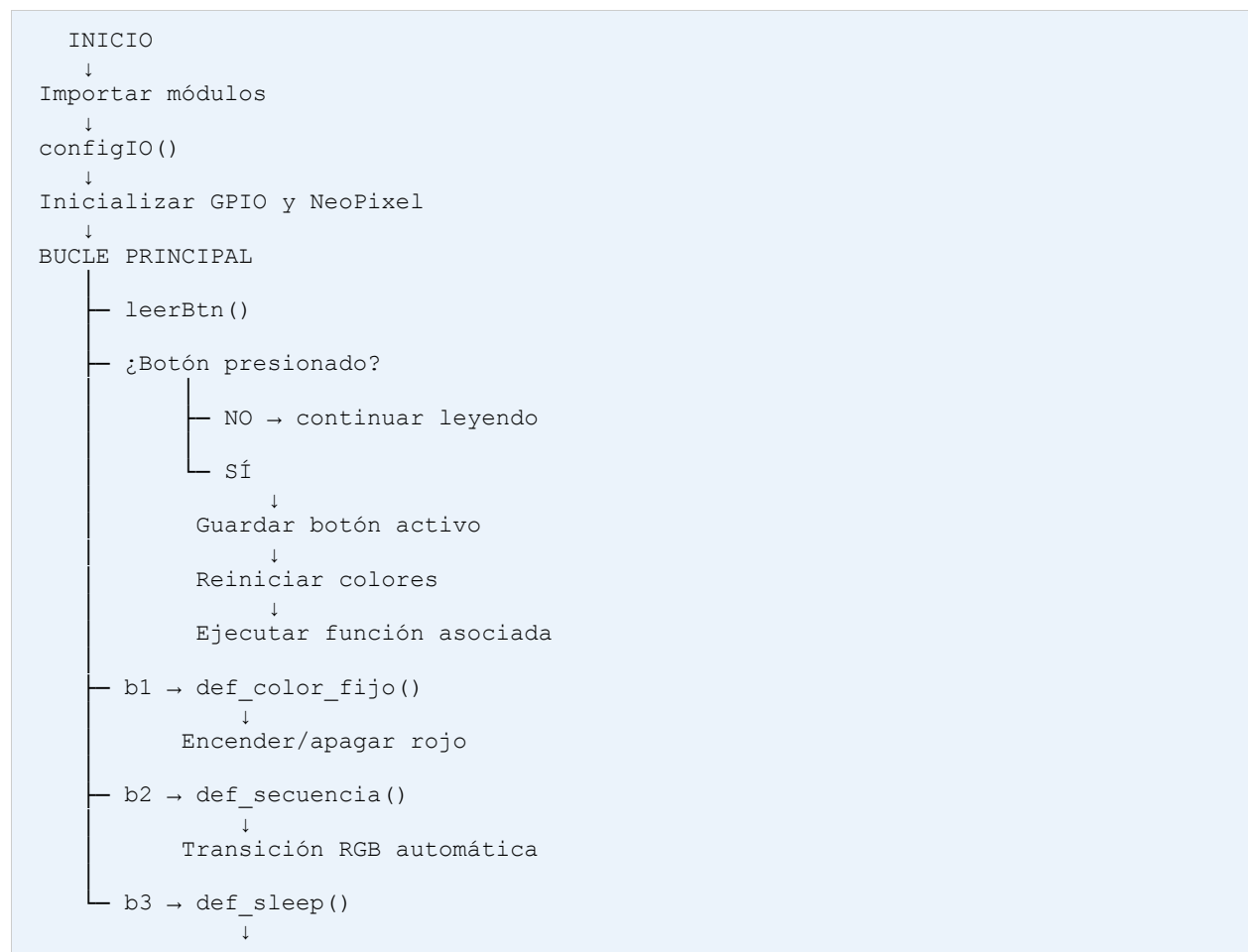
```
time.sleep_ms(100)
```

La función `sleep_ms()` suspende la ejecución durante 100 milisegundos. Este retardo reduce la velocidad de ejecución del bucle principal, evita cambios excesivamente rápidos en la secuencia de colores y disminuye la carga de procesamiento asociada al polling continuo de los botones.

En conjunto, este bucle constituye el núcleo operativo del sistema, ya que coordina la lectura de entradas, la selección de modos de funcionamiento y la ejecución de las acciones correspondientes sobre el LED RGB y el modo de bajo consumo del ESP32-C6.

5. Flujo General del Sistema

El siguiente diagrama describe el flujo de ejecución completo del firmware desde el arranque:



```
Apagar LED
  ↓
lightsleep()
  ↓
Despertar
  ↓
Regresar al bucle
```

6. Conceptos Clave

GPIO (General Purpose Input/Output)

Los GPIO son pines digitales programables del ESP32-C6 que pueden configurarse como entradas o salidas. En esta práctica se utilizan como entradas para la lectura de los pulsadores y como salida para la transmisión de datos hacia el LED NeoPixel.

Resistencias PULL-UP y PULL-DOWN

Las resistencias internas PULL-UP y PULL-DOWN permiten definir un estado lógico estable cuando un botón no está siendo presionado.

- **PULL-UP:** mantiene el pin en nivel lógico HIGH (1) en reposo. Al presionar el botón, el pin cambia a LOW (0).
- **PULL-DOWN:** mantiene el pin en nivel lógico LOW (0) en reposo. Al presionar el botón, el pin cambia a HIGH (1).

En este proyecto:

- B1 y B2 utilizan configuración PULL-DOWN.
- B3 utiliza configuración PULL-UP.

Polling

El polling es una técnica de supervisión continua mediante la cual el procesador consulta periódicamente el estado de una entrada para detectar cambios.

En esta práctica, la función leerBtn() realiza un barrido constante de los botones para determinar cuál se encuentra presionado. A diferencia de las interrupciones (IRQ), el procesador debe consultar repetidamente el estado de las entradas.

NeoPixel

NeoPixel es una familia de LEDs RGB direccionables basados generalmente en el controlador WS2812. Cada LED integra internamente los canales rojo, verde y azul junto con la lógica necesaria para recibir datos seriales desde un único pin.

7. Dependencias y Bibliotecas

Módulo/Biblioteca	Origen	Uso en el proyecto
machine.Pin	MicroPython stdlib	Configuración y lectura de GPIO
machine.lightsleep	MicroPython stdlib	Modo de bajo consumo del ESP32-C6
time	MicroPython stdlib	Retardos mediante sleep_ms()
neopixel	MicroPython stdlib	Control del LED RGB

8. Conclusiones

Esta práctica permitió implementar un sistema embebido interactivo basado en el ESP32-C6, integrando entradas digitales, control de iluminación RGB y mecanismos de ahorro energético dentro de una misma aplicación.

Entre los aspectos más relevantes destacan:

- El uso de GPIO configurados con resistencias internas PULL-UP y PULL-DOWN simplifica el diseño de hardware y garantiza lecturas estables de los pulsadores.
- La técnica de polling permite detectar el estado de múltiples entradas mediante una implementación sencilla y fácil de comprender.
- El LED NeoPixel demuestra cómo controlar colores RGB utilizando una única línea de datos, reduciendo significativamente la cantidad de conexiones requeridas.
- La implementación de una máquina de estados mediante la variable fase facilita la generación de secuencias complejas de colores de manera organizada y escalable.
- La asociación de botones y funciones mediante diccionarios proporciona una estructura modular que simplifica el mantenimiento y futuras ampliaciones del firmware.
- El modo LIGHTSLEEP constituye una herramienta eficiente para reducir el consumo energético del sistema cuando no se requiere procesamiento activo.

En conjunto, la práctica demuestra la integración de periféricos de entrada, dispositivos de salida inteligentes y técnicas básicas de gestión de energía sobre una plataforma ESP32-C6 programada en MicroPython, consolidando conceptos fundamentales del desarrollo de sistemas embebidos.

9. Referencias

- MicroPython, *MicroPython Documentation*. Disponible en: [MicroPython Documentation](#)
- MicroPython, *machine — functions related to the hardware*. Disponible en: [MicroPython machine module](#)
- MicroPython, *NeoPixel driver documentation*. Disponible en: [MicroPython NeoPixel Documentation](#)
- [Espressif Systems Official Website](#), *ESP32-C6 Series Datasheet*.
- ESP32-C6 Documentation
- Worldsemi, *WS2812B Intelligent Control LED Datasheet*. Disponible en: [WS2812B Datasheet Archive](#)

10. Anexos

```
# Importa la clase Pin para la configuración de GPIO
# e importa lightsleep para habilitar el modo de bajo consumo del ESP32-C6
from machine import Pin, lightsleep

# Importa la librería time para generar retardos temporales
import time

# Importa la librería NeoPixel para controlar LEDs RGB direccionables
import neopixel

# Diccionario utilizado para almacenar las referencias a los pines
# configurados durante la inicialización del hardware
pines = {}

# Diccionario que contiene la configuración de cada pulsador:
# pin    -> GPIO asociado
# pull   -> resistencia interna utilizada
# activo -> nivel lógico que representa una pulsación válida
btns = {
    "b1": {"pin": 3, "pull": Pin.PULL_DOWN, "activo": 1},
    "b2": {"pin": 4, "pull": Pin.PULL_DOWN, "activo": 1},
    "b3": {"pin": 1, "pull": Pin.PULL_UP, "activo": 0},
}

# Lista que almacena los valores RGB actuales del NeoPixel:
# colores[0] = rojo
# colores[1] = verde
# colores[2] = azul
colores = [0, 0, 0]

# Función encargada de activar o desactivar el color rojo fijo
def def_color_fijo():

    # Si el canal rojo vale 0 se coloca en 255.
    # Si ya vale 255 se regresa a 0.
    colores[0] = 0 if colores[0] else 255

    # Actualiza el color del NeoPixel
    pines["rgb"][0] = colores

    # Envía los datos al LED para reflejar el cambio
    pines["rgb"].write()

# Variable global que almacena la fase actual de la secuencia RGB
fase = 0

# Función encargada de generar una transición continua
# entre los distintos colores del espectro RGB
```

```

def def_secuencia():

    # Permite modificar la variable global fase
    global fase

    # Incremento aplicado en cada actualización
    step = 10

    # Fase 0: transición Rojo -> Amarillo
    if fase == 0:
        colores[1] += step

        if colores[1] >= 255:
            colores[1] = 255
            fase = 1
    # Fase 1: transición Amarillo -> Verde
    elif fase == 1:
        colores[0] -= step

        if colores[0] <= 0:
            colores[0] = 0
            fase = 2
    # Fase 2: transición Verde -> Cian
    elif fase == 2:
        colores[2] += step

        if colores[2] >= 255:
            colores[2] = 255
            fase = 3
    # Fase 3: transición Cian -> Azul
    elif fase == 3:
        colores[1] -= step

        if colores[1] <= 0:
            colores[1] = 0
            fase = 4
    # Fase 4: transición Azul -> Magenta
    elif fase == 4:
        colores[0] += step

        if colores[0] >= 255:
            colores[0] = 255
            fase = 5
    # Fase 5: transición Magenta -> Rojo
    elif fase == 5:
        colores[2] -= step

        if colores[2] <= 0:
            colores[2] = 0
            fase = 0

    # Actualiza el color calculado en el NeoPixel
    pines["rgb"][0] = colores

    # Envía los nuevos valores RGB al LED
    pines["rgb"].write()
# Función encargada de apagar el LED e ingresar
# al modo de bajo consumo LIGHTSLEEP
def def_sleep():

    # Permite modificar la variable global colores
    global colores

    # Apaga completamente el NeoPixel
    colores = [0, 0, 0]

    # Actualiza el buffer interno del LED
    pines["rgb"][0] = colores

    # Envía los cambios al NeoPixel
    pines["rgb"].write()

```

```

# Coloca al ESP32-C6 en modo de bajo consumo
lightsleep()

# Diccionario que relaciona cada botón con la función
# que debe ejecutarse cuando dicho botón es seleccionado
funciones = {
    "b1": def_color_fijo,
    "b2": def_secuencia,
    "b3": def_sleep
}

# Función encargada de inicializar todos los GPIO del sistema
def configIO():
    # Recorre todas las configuraciones almacenadas en btns
    for nombre, cfg in btns.items():
        # Configura cada GPIO como entrada digital
        # utilizando la resistencia interna especificada
        pines[nombre] = Pin(
            cfg["pin"],
            Pin.IN,
            cfg["pull"]
        )
    # Configura un NeoPixel conectado al GPIO 8
    # El segundo parámetro indica que existe un solo LED
    pines["rgb"] = neopixel.NeoPixel(Pin(8), 1)

# Ejecuta la configuración inicial del hardware
configIO()

# Función encargada de identificar qué botón se encuentra presionado
def leerBtn():
    # Recorre todos los botones configurados
    for nombre, cfg in btns.items():

        # Compara el estado actual del GPIO con el nivel lógico
        # definido como activación para dicho pulsador
        if pines[nombre].value() == cfg["activo"]:

            # Retorna el nombre del botón detectado
            return nombre

    # Si ningún botón fue presionado retorna None
    return None

# Variable utilizada para almacenar el último botón detectado
btn = None

# Bucle principal del sistema
while True:
    # Realiza la lectura de los pulsadores
    lectura = leerBtn()

    # Si se detectó un botón válido
    if lectura is not None:

        # Guarda el nombre del botón detectado
        btn = lectura

        # Reinicia los valores RGB antes de ejecutar
        # la función correspondiente
        colores = [0, 0, 0]

    # Verifica que el botón almacenado exista
    # dentro de la tabla de funciones
    if btn in btns:

        # Ejecuta la función asociada al botón
        funciones[btn]()

        # Introduce una pausa de 100 ms para reducir
        # la velocidad de ejecución del bucle
        time.sleep_ms(100)

```